Cash Positions and Obligation Distributions

Setting

There are ten firms in the economy that transact with each other and with consumers.

- Firms trade with each other using invoices that are issued today and paid on a future due date.
- When an invoice is issued, the seller has an accounts-receivable claim and the buyer has an accounts-payable liability.
- When an invoice is paid (on the due date), the seller receives cash and the buyer pays cash.
- Firms also sell to consumers. These sales are recorded as receipts with immediate payment on the sale date.
- Firms have operational expenses (and possibly equity payouts) with immediate payment on the expense date.
- Firms borrow using two instruments: term loans and lines of credit. Borrowing provides cash at origination and creates a repayment outflow at maturity.
- Interest rates are annualized. Repayment uses simple interest: amount due $=$ principal $\times \left(1 + \frac{\text{rate}}{100} \times \frac{\text{maturity days}}{365}\right)$.
- Assume all invoices and debt are paid on time.

Data documentation

All dates below are interpreted as calendar dates.

Credit dataset (`df_credit`). is loan-level (one row per security/loan).

- `id_borrower`: firm identifier (borrower).
- `id_security`: security identifier.
- `borrow_date`: origination date (in the script this is already a date column).
- `loan_rate`: annualized interest rate in percent (e.g., 12.5 means 12.5%).
- `loan_borrowed`: principal amount borrowed.
- `maturity`: maturity in days.
- `loan_type`: `"Term Loan"` or `"Line of Credit"`.

Invoice dataset (`df_invoices`). has one row per buyer–seller invoice.

- `hash_invoice`: invoice identifier.
- `id_seller`: firm identifier of the seller.
- `id_buyer`: firm identifier of the buyer.
- `issue_date`: contract date.
- `due_date`: payment due date.
- `maturity`: contractual maturity in days (from issue to due date).
- `payment`: contracted invoice amount.

Receipts dataset (`df_receipts`). has consumer sales receipts (immediate cash inflows).

- `hash_receipt`: receipt identifier.
- `id_seller`: firm identifier of the seller.
- `sale_date`: date of sale and payment.
- `payment`: cash received on `sale_date`.

Expenses dataset (`df_expenses`). has immediate cash outflows.

- `hash_expense`: expense identifier.
- `id_firm`: firm identifier.
- `expense_date`: date of payment.
- `payment`: cash paid on `expense_date`.

Firms dataset (`df_firms`). has one row per firm (firm-level attributes).

- `id_firm`: firm identifier (1–10); the join key for `df_invoices`, `df_receipts`, `df_expenses`, and `df_credit`.
- `industry`: sector classification (`"Manufacturing"`, `"Retail"`, `"Services"`, `"Finance"`).
- `founding_year`: year the firm was founded.
- `start_cash`: cash on hand at the start of 2023 (in USD).

Instructions

Produce Table 1 and Figures 1–6 described below.

- Your code should run end-to-end from the project root using relative paths (no machine-specific absolute paths).
- Unless explicitly requested otherwise, report USD magnitudes in **thousands** on plotted y-axes.

Table 1: Annual Trade Credit Summary (all firms)

For each firm, compute the following using `df_invoices` for calendar year 2023 (i.e., where `issue_date` falls in 2023):

- `total_ar`: total accounts receivable = sum of `payment` on invoices where the firm is the *seller*.
- `total_ap`: total accounts payable = sum of `payment` on invoices where the firm is the *buyer*.
- `net_trade_credit`: `total_ar` − `total_ap`.
- `n_customers`: number of distinct buyers the firm sold to.
- `n_suppliers`: number of distinct sellers the firm bought from.

Then merge in firm-level attributes (`industry`, `founding_year`) from `df_firms` using a left join on `id_firm`. Output a formatted table sorted by `net_trade_credit` descending. Every firm (1–10) must appear in the output even if it had zero activity on one or both sides.

Figure 1: Cash on Hand

Construct daily cash on hand for each firm in 2023 by netting all cash events at the firm-day level. Each firm's starting cash is `start_cash` from `df_firms`. Define the cumulative sum of net daily cash changes as the cash position at each point in time. Plot Firms 1–3 as solid lines over the 2023 calendar year.

- x-axis: date (2023-01-01 to 2023-12-31)
- y-axis: cash on hand in thousands of USD
- line colors: Firm 1 = brown, Firm 2 = red, Firm 3 = navy

Note that the cash timeline is restricted to the 2023 calendar year. Throughout this test, the USD amounts are solely illustrative, smaller than for real-world firms.

Figures 2–3: Obligation maturity distribution (Firm 5 and Firm 6)

Using the stacked-bar plotting helper, produce two maturity-distribution figures of obligations for a specific firm. For each firm, define obligations as:

- debt due: term loans and lines of credit for that firm, summed by contractual maturity bin using `amount_due`
- payables: invoice payments where that firm is the *buyer*, summed by contractual maturity bin using the invoice payment

Maturity bins for obligations are based on contractual maturity in days:

$$\{0\text{–}30,\ 31\text{–}60,\ 61\text{–}90,\ 91\text{–}120,\ 121\text{–}180,\ \text{Above } 180\}.$$

Use the following order for the stacked bars:

- bottom: Term Loan (gray)
- middle: Line of Credit (navy)
- top: Payables (brown)

Axes/labels:

- x-axis title: Contractual Maturity
- y-axis: obligations in thousands of USD
- legend on the right with title: Repayment Type

Figures 4–5: Interest rate distribution (credit market)

Create interest-rate distributions for term loans versus lines of credit over the full sample.

- Round `loan_rate` to the nearest integer.
- Bin rates into:
$$\{0\text{–}10,\ 11\text{–}20,\ 21\text{–}30,\ 31\text{–}40,\ 41\text{–}50,\ \text{Above } 50\}.$$
- For each bin and loan type, compute:
  - number of loans (`num_loans`)
  - total principal borrowed (`total_borrowed`)
- Plot stacked bars by rate bin with legend on the right (Term Loan in gray, Line of Credit in navy).

Figure numbering convention:

- Figure 4: stacked bars for `num_loans`
- Figure 5: stacked bars for `total_borrowed`

Figure 6: Rolling Outstanding Payables vs. Receivables (Firm 3)

For each calendar date t in 2023, compute Firm 3's *stock* of outstanding trade credit:

- **Outstanding AR**: sum of `payment` for all invoices where `id_seller` $= 3$, `issue_date` $\leqslant$ t, and `due_date` $>$ t (invoices that have been issued but not yet collected as of date t).

- **Outstanding AP**: sum of `payment` for all invoices where `id_buyer` $=$ 3, `issue_date` $\leqslant$ t, and `due_date` $>$ t (invoices that have been issued but not yet paid as of date t).

Important: some invoices may be issued before 2023 and/or due after 2023. For any 2023 date t, include them whenever the outstanding condition (`issue_date` $\leqslant$ t $<$ `due_date`) holds. Plot both series as solid lines on the same axes over the full 2023 calendar year.

- x-axis: date (2023-01-01 to 2023-12-31)
- y-axis: outstanding amount in thousands of USD
- line colors: Outstanding AR = navy, Outstanding AP = brown

Hint: think about how to evaluate, for every date in a date sequence, which invoices are simultaneously post-issue and pre-due. A cross-join between the date sequence and the invoice data is one clean approach.